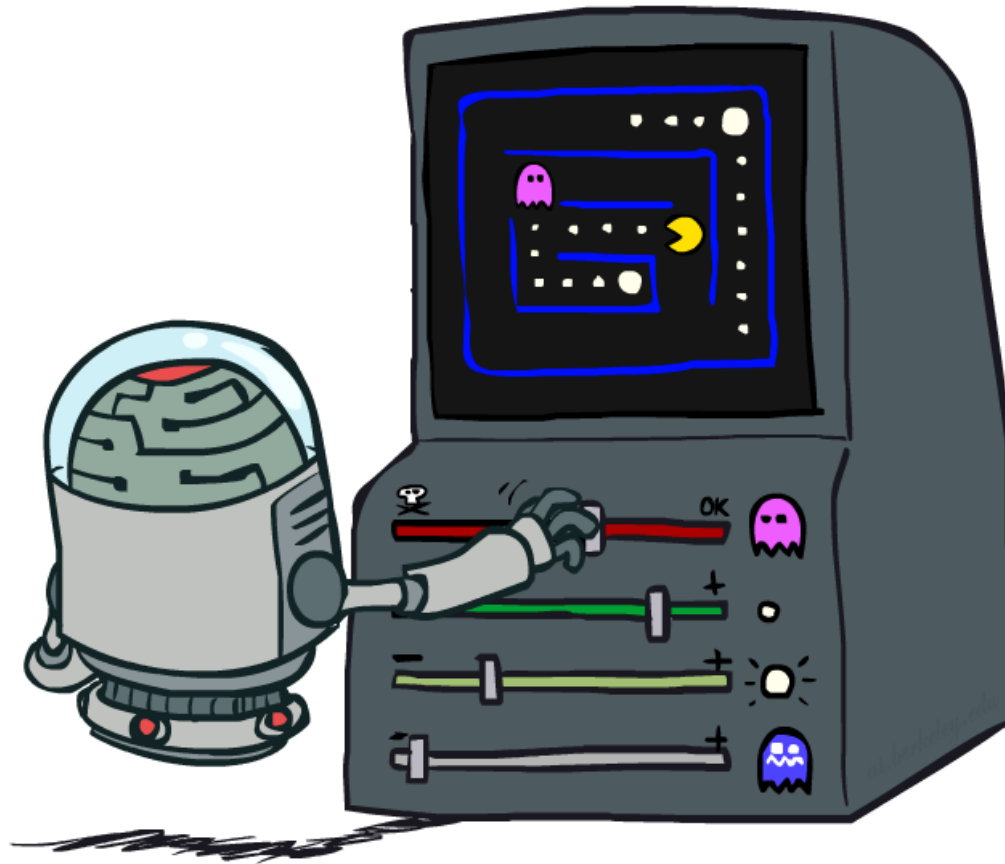


CS 6300: Artificial Intelligence

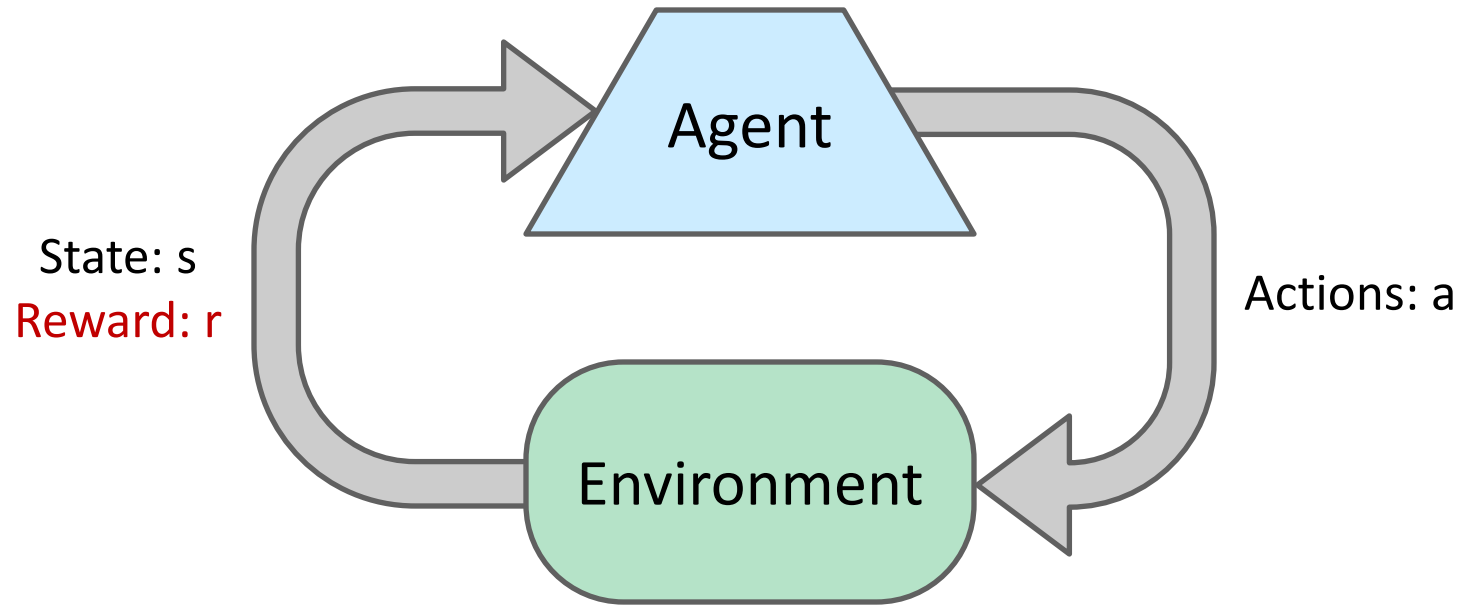
Reinforcement Learning II: Function Approximation



Instructor: Daniel Brown --- University of Utah

[Based on slides created by Dan Klein and Pieter Abbeel <http://ai.berkeley.edu>.]

Reinforcement Learning



- **Basic idea:**

- Receive feedback in the form of **rewards**
- Agent's utility is defined by the reward function
- Must (learn to) act so as to **maximize expected rewards**
- All learning is based on observed samples of outcomes!

Reinforcement Learning

- We still assume an MDP:
 - A set of states $s \in S$
 - A set of actions (per state) A
 - A model $T(s,a,s')$
 - A reward function $R(s,a,s')$
- Still looking for a policy $\pi(s)$
- New twist: don't know T or R , so must try out actions
- Big idea: Compute all averages over T using sample outcomes



The Story So Far: MDPs and RL

Known MDP: Offline Solution

Goal

Compute V^* , Q^* , π^*

Evaluate a fixed policy π

Technique

Value / policy iteration

Policy evaluation

Unknown MDP: Model-Based

Goal

Compute V^* , Q^* , π^*

Evaluate a fixed policy π

Technique

VI/PI on approx. MDP

PE on approx. MDP

Unknown MDP: Model-Free

Goal

Compute V^* , Q^* , π^*

Evaluate a fixed policy π

Technique

Q-learning

Value Learning

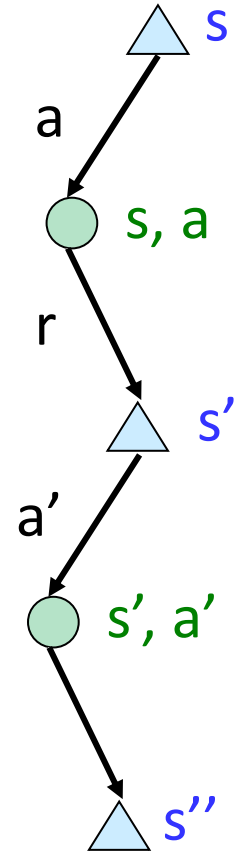
Model-Free Learning

- Model-free (temporal difference) learning

- Experience world through episodes

$$(s, a, r, s', a', r', s'', a'', r'', s'''' \dots)$$

- Update estimates each transition (s, a, r, s')
- Over time, updates will mimic Bellman updates



Q-Learning

- We'd like to do Q-value updates to each Q-state:

$$Q_{k+1}(s, a) \leftarrow \sum_{s'} T(s, a, s') \left[R(s, a, s') + \gamma \max_{a'} Q_k(s', a') \right]$$

- But can't compute this update without knowing T, R
- Instead, compute average as we go
 - Receive a sample transition (s,a,r,s')
 - This sample suggests $Q(s, a) \approx r + \gamma \max_{a'} Q(s', a')$
 - But we want to average over results from (s,a) (Why?)
 - So keep a running average

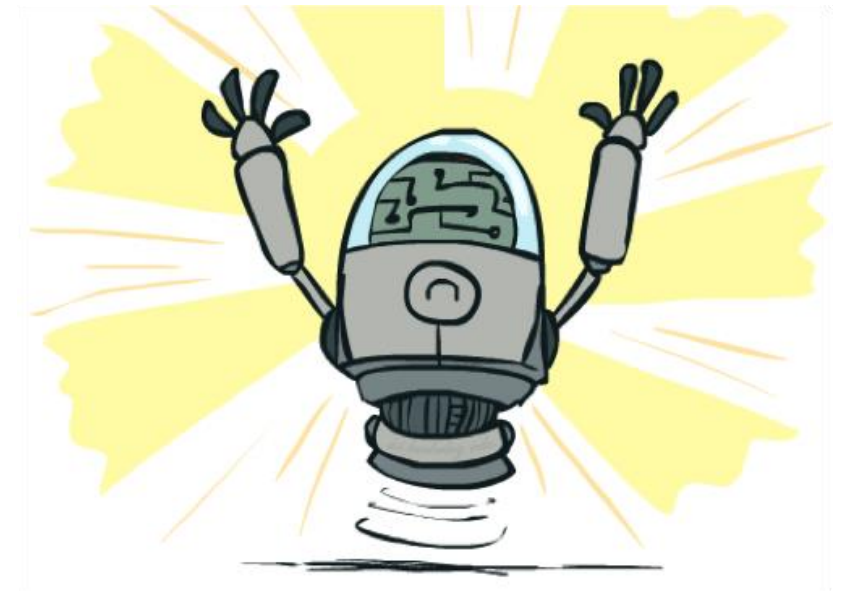
$$Q(s, a) \leftarrow (1 - \alpha)Q(s, a) + (\alpha) \left[r + \gamma \max_{a'} Q(s', a') \right]$$

$$Q(s, a) \leftarrow Q(s, a) + \alpha(r + \gamma \max_{a'} Q(s', a') - Q(s, a))$$

Useful alternate form of update for Q-learning. We want to push the Q-value towards the sample!

Q-Learning Properties

- Amazing result: Q-learning converges to optimal policy -- even if you're acting suboptimally!
- This is called **off-policy learning**
- Caveats:
 - You have to explore enough
 - You have to eventually make the learning rate small enough
 - ... but not decrease it too quickly
 - Basically, in the limit, it doesn't matter how you select actions (!)



When might RL be a good tool for your problem?

When might RL be a good tool for your problem?

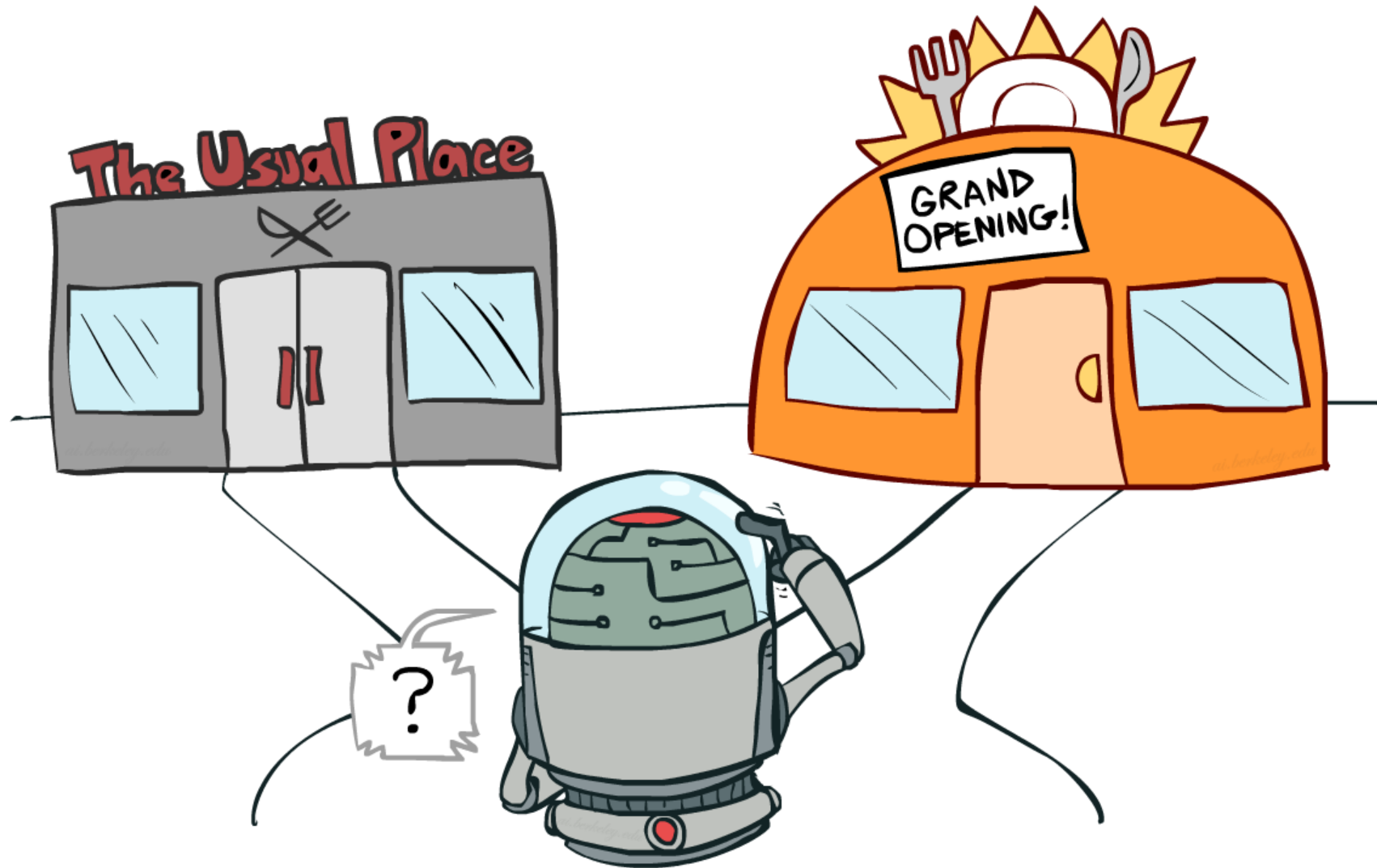
- Is your problem a sequential decision making problem?
- Are there “actions” that effect the next “state”?
- Do you know the rules of these effects?
- Can you write down a clear objective/score/reward/cost?
- Do you have a simulator?
- Lots of examples of sequences of decisions and their long-term consequences?
- Is it unclear what to do in each state? Exploration required?
- Are you looking for unique/creative/super-human solutions?

When might RL not be a good tool?

When might RL not be a good tool?

- Single step or static problem
- No clear reward signal.
- Reward signal is unavailable or very hard to write down.
- Well-known model of the environment.
- Deterministic environment
- Low-tolerance for exploration and trial and error
- No need for adaptive or novel solutions. The goal is to perform the task in a very predictable way.

Exploration vs. Exploitation



How to Explore?

- Several schemes for forcing exploration
 - Simplest: random actions (ϵ -greedy)
 - Every time step, flip a coin
 - With (small) probability ϵ , act randomly
 - With (large) probability $1-\epsilon$, act on current policy
 - Problems with random actions?
 - You do eventually explore the space, but keep thrashing around once learning is done
 - One solution: lower ϵ over time
 - Another solution: Boltzmann (softmax) exploration
 - Another solution: exploration functions

Exploration Functions

- When to explore?

- Random actions: explore a fixed amount
- Better idea: explore areas whose badness is not (yet) established, eventually stop exploring

- Exploration function

- Takes a value estimate u and a visit count n , and returns an optimistic utility, e.g. $f(u, n) = u + k/n$

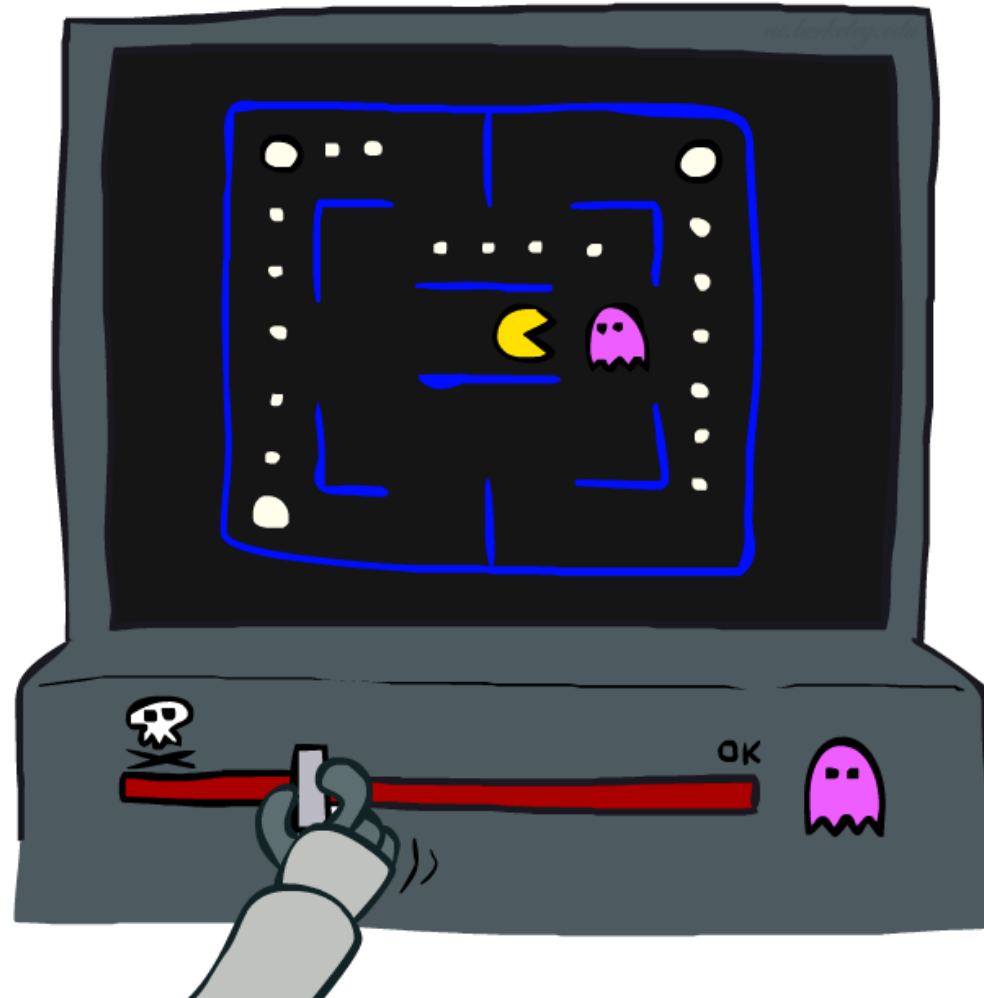
Regular Q-Update: $Q(s, a) \leftarrow_{\alpha} R(s, a, s') + \gamma \max_{a'} Q(s', a')$

Modified Q-Update: $Q(s, a) \leftarrow_{\alpha} R(s, a, s') + \gamma \max_{a'} f(Q(s', a'), N(s', a'))$

- Note: this propagates the “bonus” back to states that lead to unknown states as well!

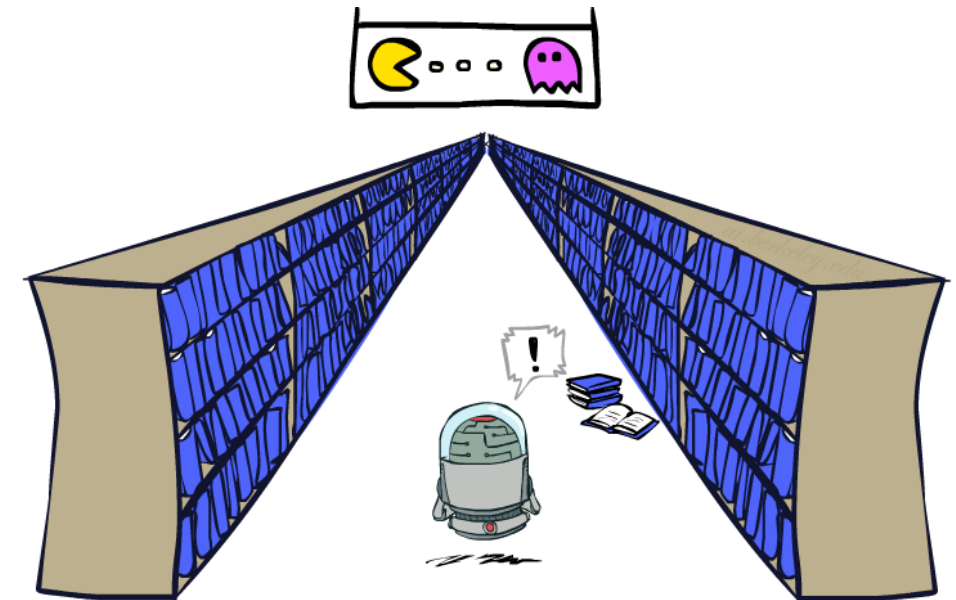
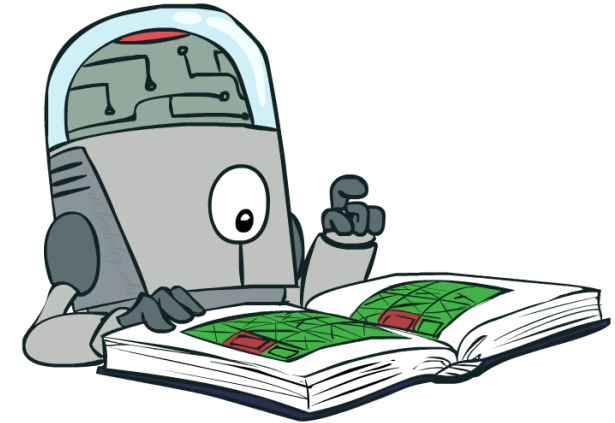


Approximate Q-Learning



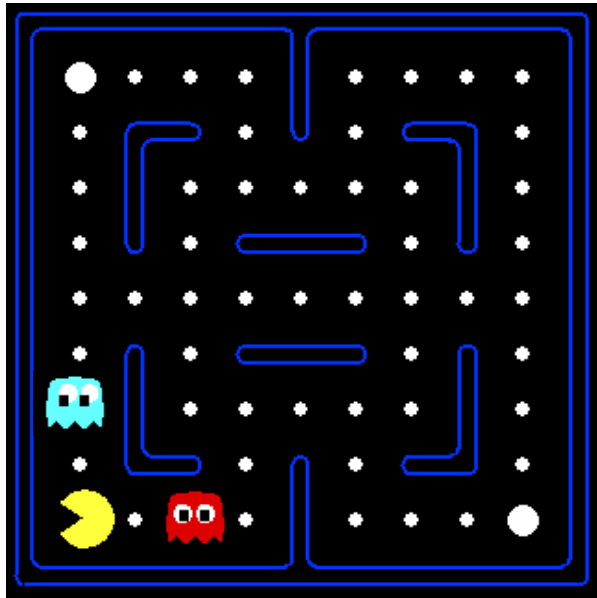
Generalizing Across States

- Basic Q-Learning keeps a table of all q-values
- In realistic situations, we cannot possibly learn about every single state!
 - Too many states to visit them all in training
 - Too many states to hold the q-tables in memory
- Instead, we want to generalize:
 - Learn about some small number of training states from experience
 - Generalize that experience to new, similar situations
 - This is a fundamental idea in machine learning, and we'll see it over and over again

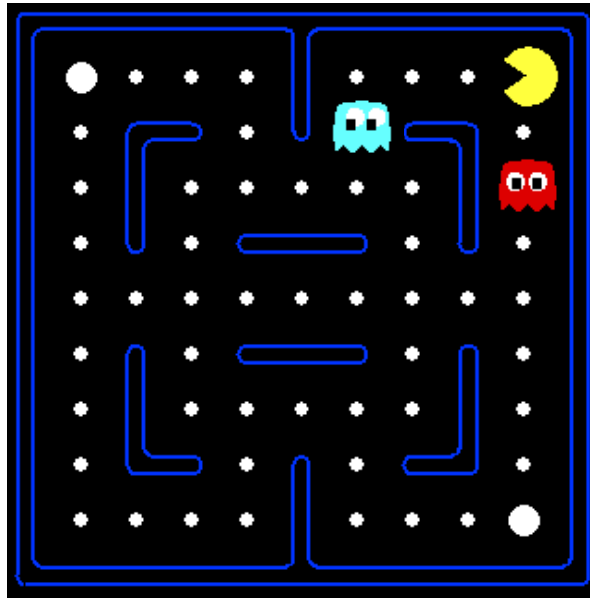


Example: Pacman

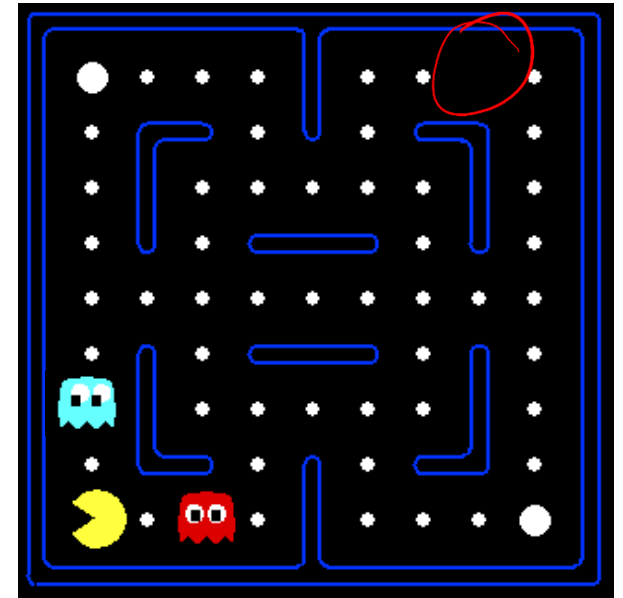
Let's say we discover through experience that this state is bad:



In naïve q-learning, we know nothing about this state:

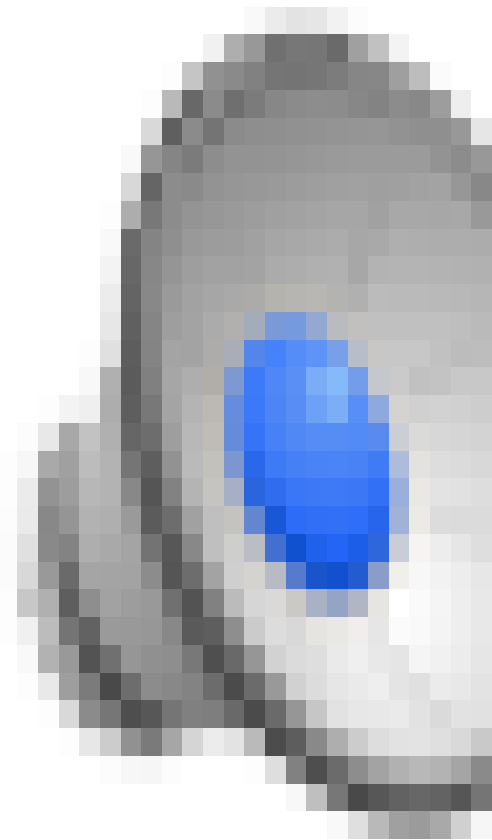


Or even this one!



Video of Demo Q-Learning Pacman – Tiny

At the
beginning of
learning

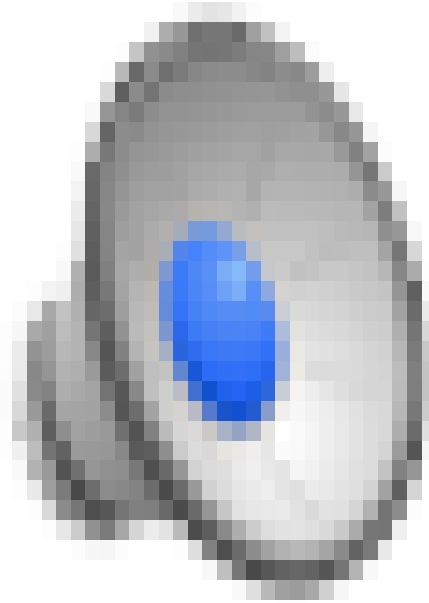


Video of Demo Q-Learning Pacman – Tiny

After 2000
episodes

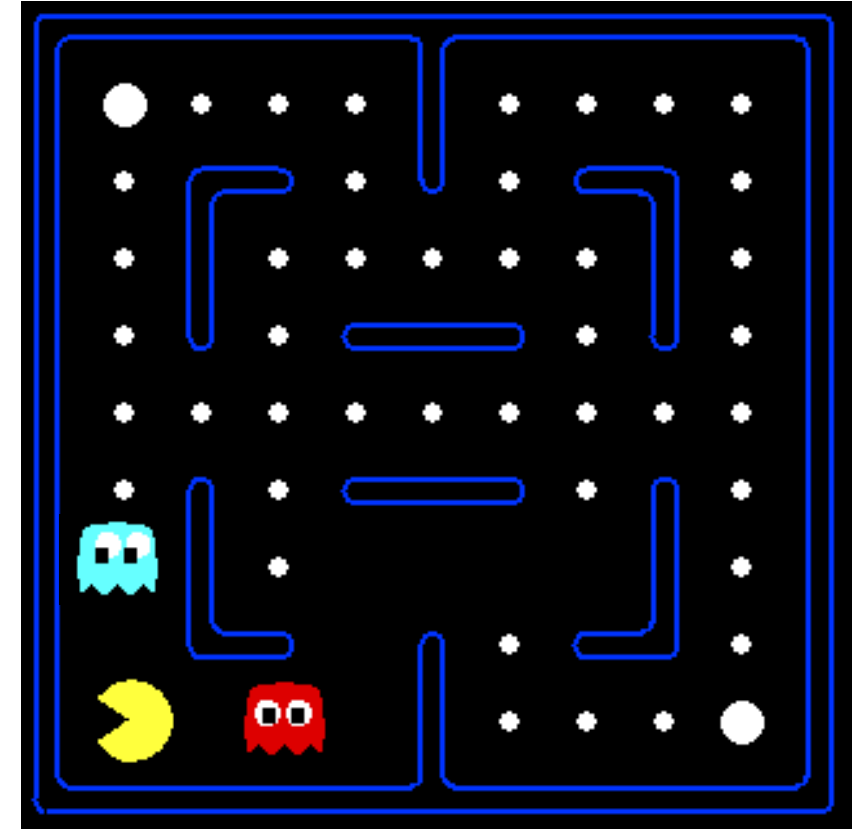


Video of Demo Q-Learning Pacman – Tricky



Feature-Based Representations

- Solution: describe a state using a vector of features (properties)
 - Features are functions from states to real numbers (often 0/1) that capture important properties of the state
 - Example features:
 - Distance to closest ghost
 - Distance to closest dot
 - Number of ghosts
 - $1 / (\text{dist to dot})^2$
 - Is Pacman in a tunnel? (0/1)
 - etc.
 - Is it the exact state on this slide?
 - Can also describe a q-state (s, a) with features (e.g. action moves closer to food)



Linear Value Functions

- Using a feature representation, we can write a q function (or value function) for any state using a few weights:

$$V(s) = w_1 f_1(s) + w_2 f_2(s) + \dots + w_n f_n(s)$$

$$Q(s, a) = w_1 f_1(s, a) + w_2 f_2(s, a) + \dots + w_n f_n(s, a)$$

- Advantage: our experience is summed up in a few powerful numbers
- Disadvantage: states may share features but actually be very different in value!

Approximate Q-Learning

$$Q(s, a) = w_1 f_1(s, a) + w_2 f_2(s, a) + \dots + w_n f_n(s, a)$$

- Q-learning with linear Q-functions:

$$\text{transition} = (s, a, r, s')$$

$$\text{difference} = \left[r + \gamma \max_{a'} Q(s', a') \right] - Q(s, a)$$

$$Q(s, a) \leftarrow Q(s, a) + \alpha [\text{difference}]$$

$$w_i \leftarrow w_i + \alpha [\text{difference}] f_i(s, a)$$

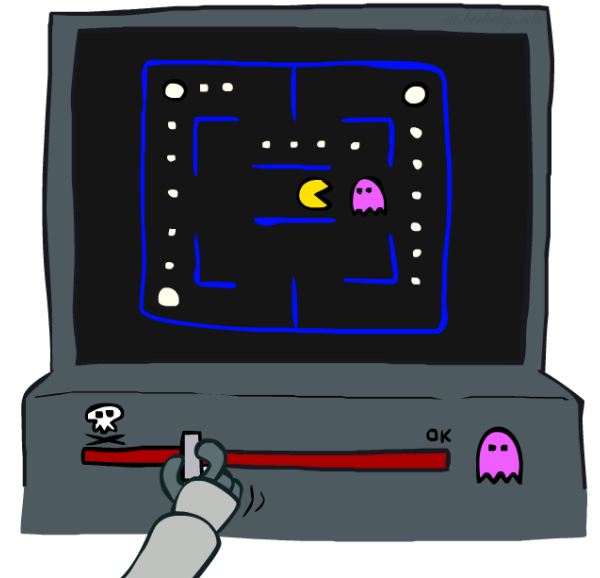
Exact Q's

Approximate Q's

- Intuitive interpretation:

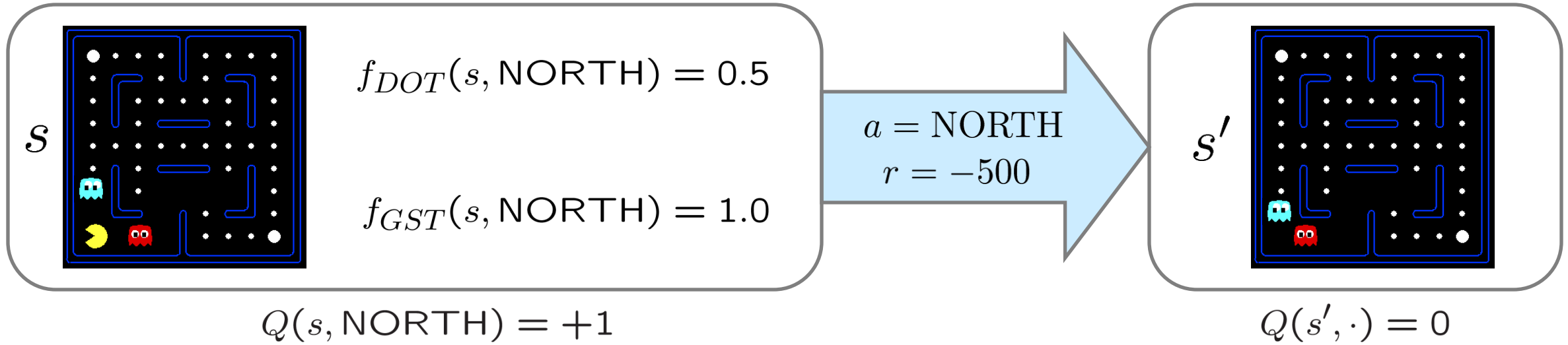
- Adjust weights of active features
- E.g., if something unexpectedly bad happens, blame the features that were on: disprefer all states with that state's features

- Formal justification: online least squares



Example: Q-Pacman

$$Q(s, a) = 4.0 f_{DOT}(s, a) - 1.0 f_{GST}(s, a)$$



$$r + \gamma \max_{a'} Q(s', a') = -500 + 0$$

difference = -501

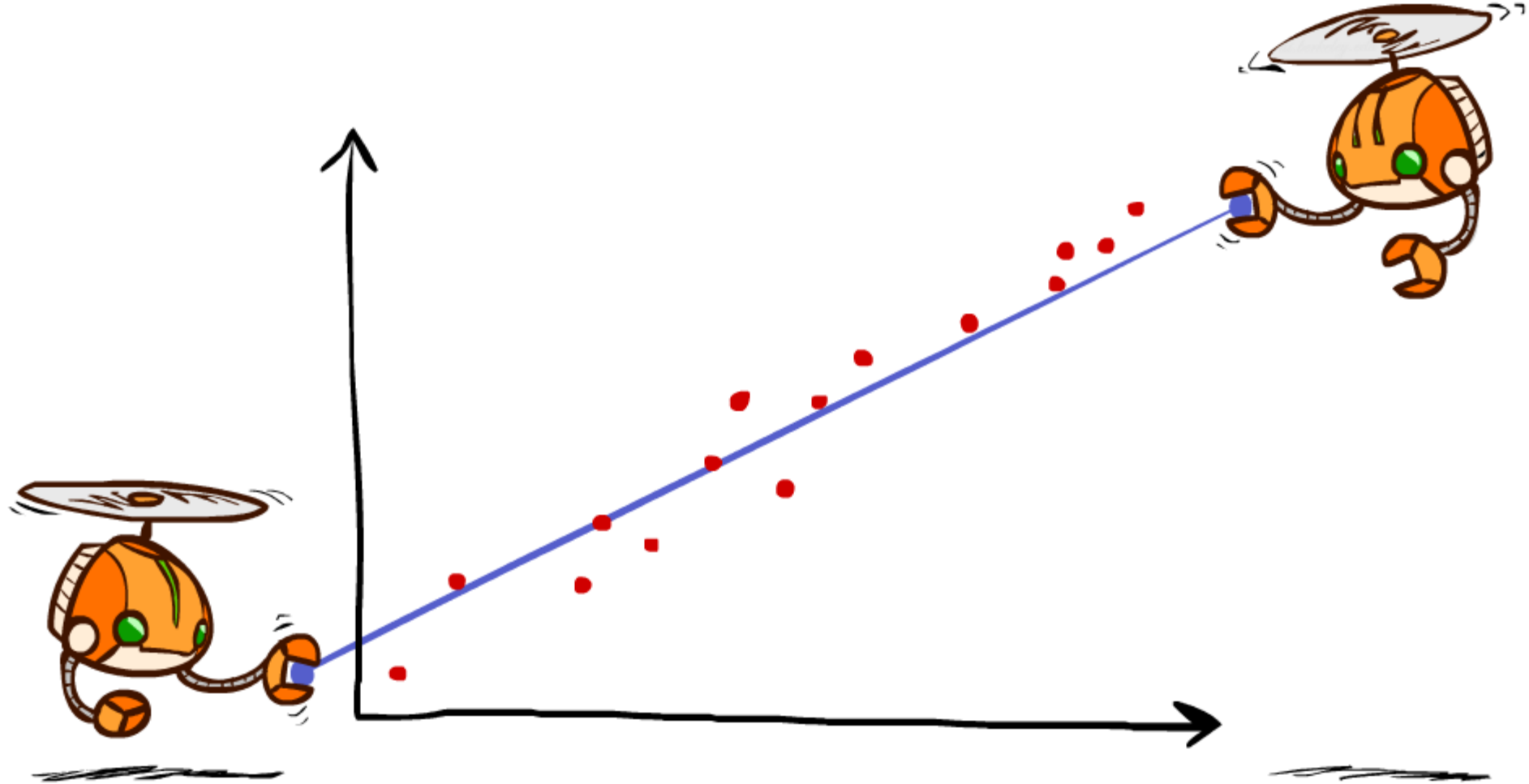
$$w_{DOT} \leftarrow 4.0 + \alpha [-501] 0.5$$
$$w_{GST} \leftarrow -1.0 + \alpha [-501] 1.0$$

$$Q(s, a) = 3.0 f_{DOT}(s, a) - 3.0 f_{GST}(s, a)$$

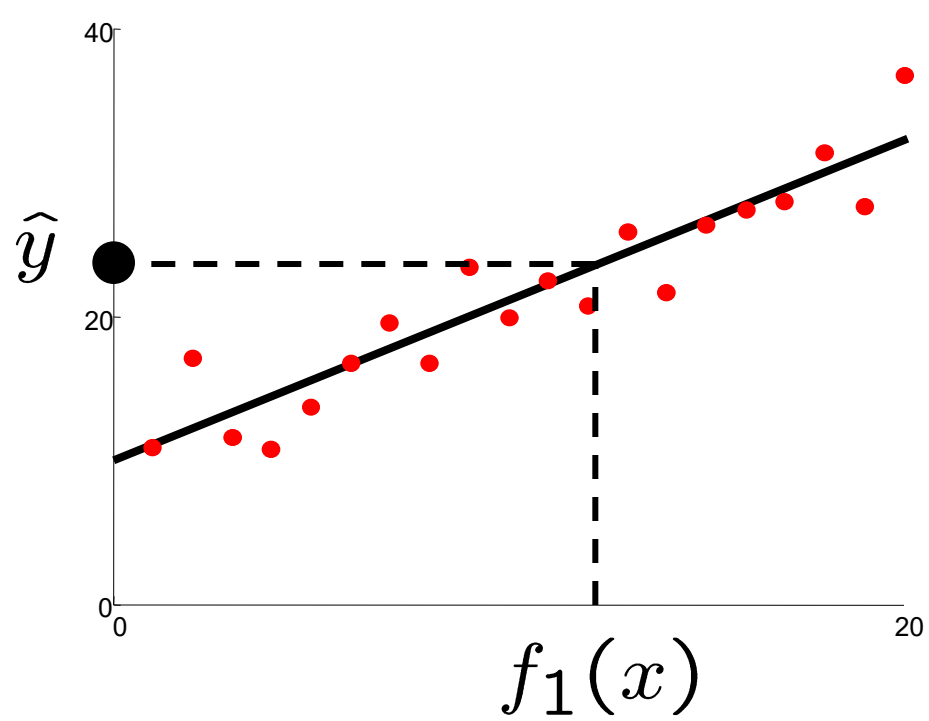
Video of Demo Approximate Q-Learning -- Pacman



Q-Learning and Linear Regression

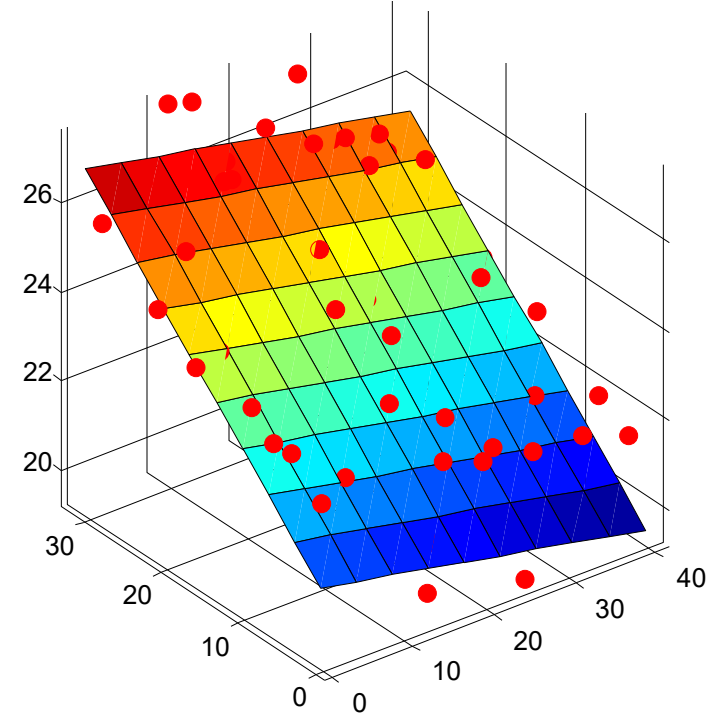


Linear Approximation: Regression



Prediction:

$$\hat{y} = w_0 + w_1 f_1(x)$$

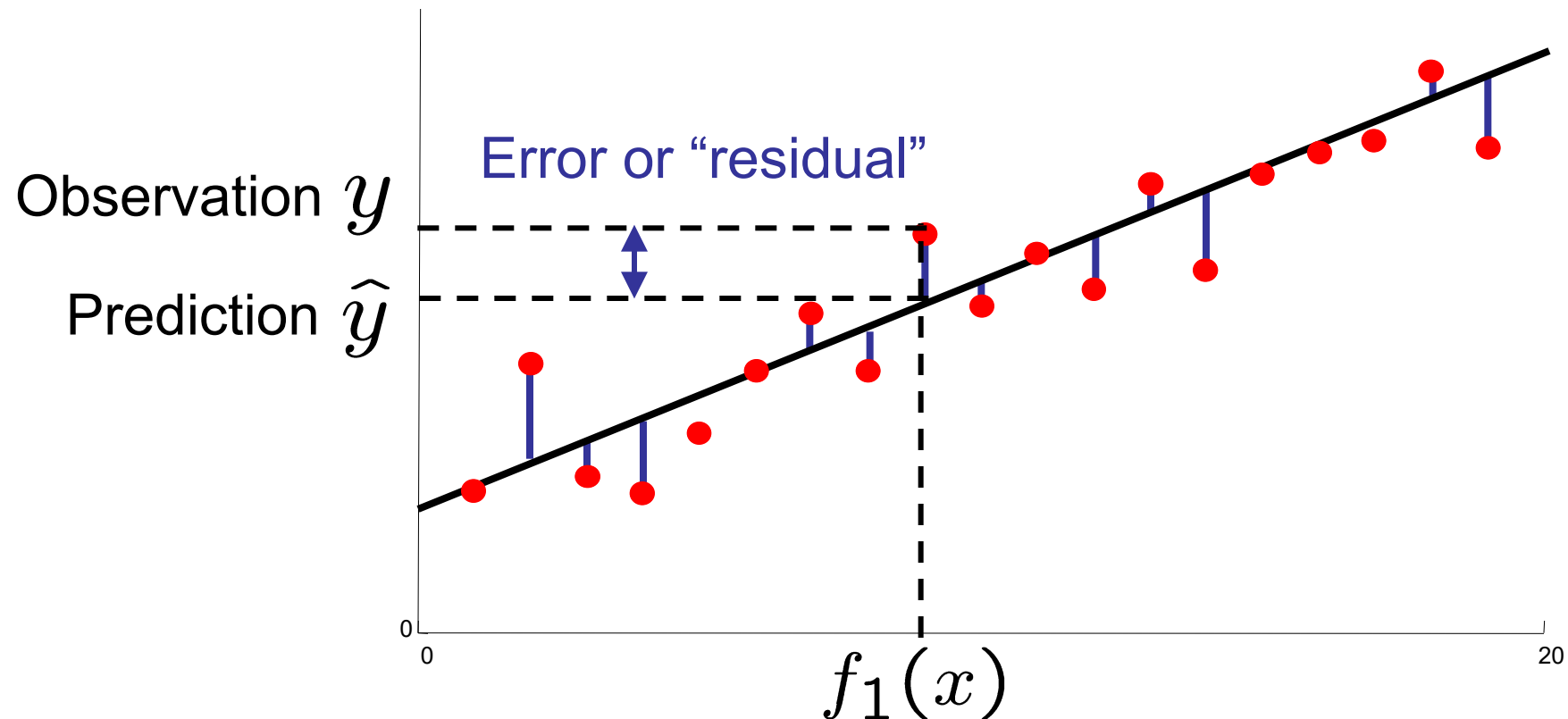


Prediction:

$$\hat{y}_i = w_0 + w_1 f_1(x) + w_2 f_2(x)$$

Optimization: Least Squares

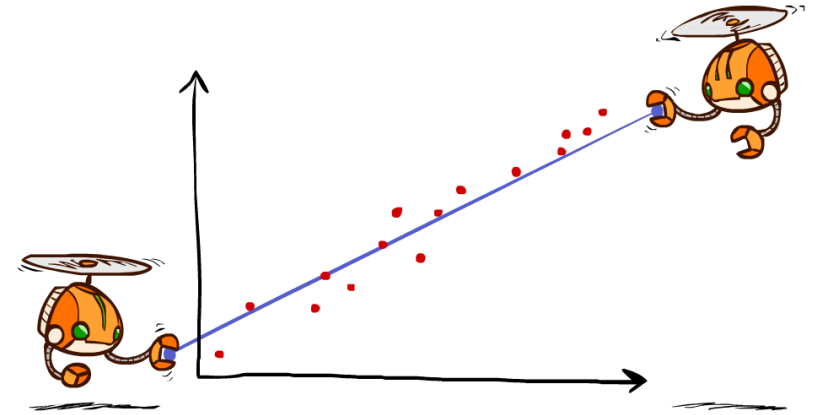
$$\text{total error} = \sum_i (y_i - \hat{y}_i)^2 = \sum_i \left(y_i - \sum_k w_k f_k(x_i) \right)^2$$



Minimizing Error

Imagine we had only one point x , with features $f(x)$, target value y , and weights w :

$$\begin{aligned}\text{error}(w) &= \frac{1}{2} \left(y - \sum_k w_k f_k(x) \right)^2 \\ \frac{\partial \text{error}(w)}{\partial w_m} &= - \left(y - \sum_k w_k f_k(x) \right) f_m(x) \\ w_m &\leftarrow w_m + \alpha \left(y - \sum_k w_k f_k(x) \right) f_m(x)\end{aligned}$$



Approximate q update explained:

$$w_m \leftarrow w_m + \alpha \left[\underbrace{r + \gamma \max_{a'} Q(s', a')}_{\text{“target”}} - \underbrace{Q(s, a)}_{\text{“prediction”}} \right] f_m(s, a)$$

“target”

“prediction”

Tabular Q-Learning is Special Case

$$w_m \leftarrow w_m + \alpha \left[\underset{\text{"target"}}{r + \gamma \max_a Q(s', a')} - \underset{\text{"prediction"}}{Q(s, a)} \right] f_m(s, a)$$

$$Q(s, a) \leftarrow (1 - \alpha)Q(s, a) + (\alpha) \left[r + \gamma \max_{a'} Q(s', a') \right]$$

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left[\underset{\text{"target"}}{r + \gamma \max_{a'} Q(s', a')} - \underset{\text{"prediction"}}{Q(s, a)} \right]$$

If feature is just an indicator for (s,a), then we recover the original tabular setting.

Non-linear function approximation

$$V(s) = w_1 f_1(s) + w_2 f_2(s) + \dots + w_n f_n(s)$$

$$Q(s, a) = w_1 f_1(s, a) + w_2 f_2(s, a) + \dots + w_n f_n(s, a)$$

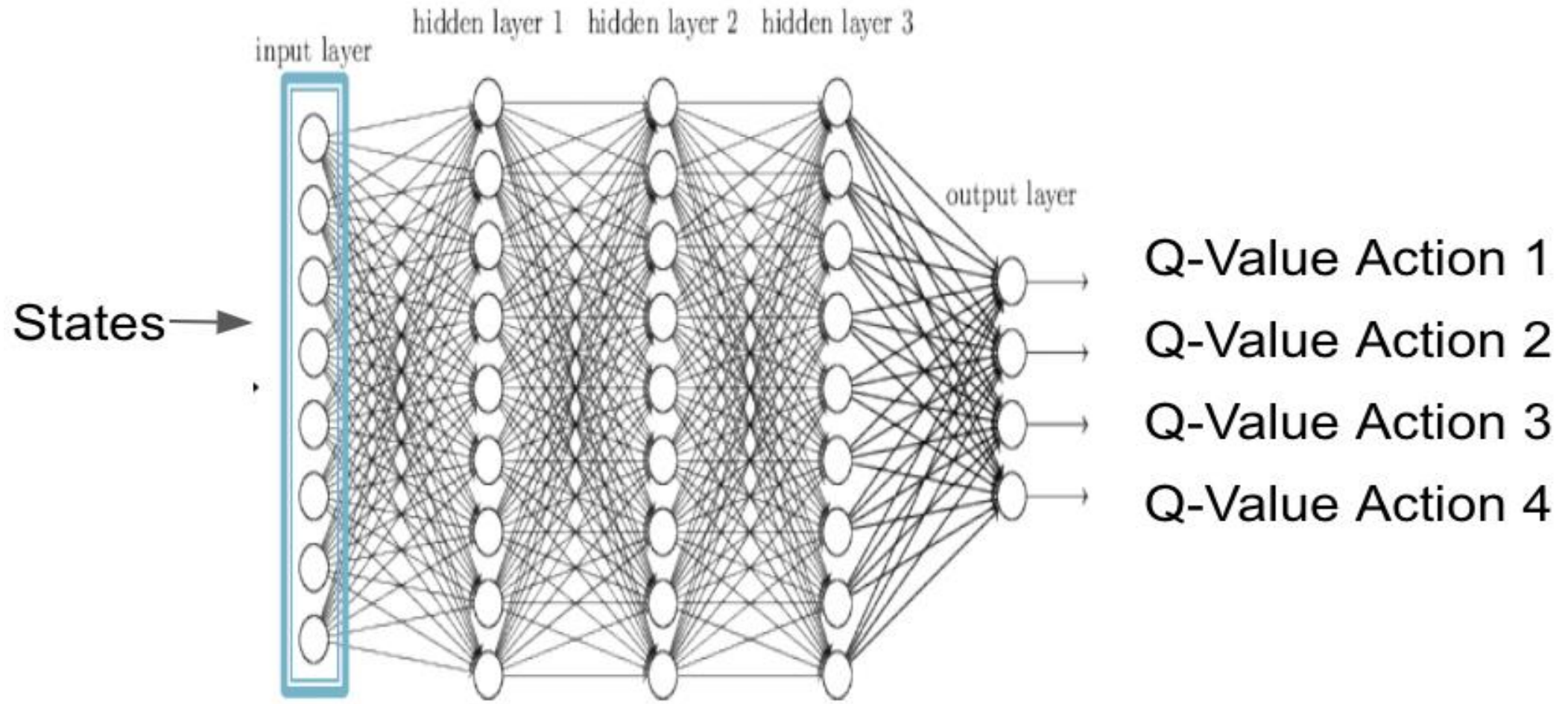
v.s.

$$V(s) = f_{\theta}(s)$$

Deep Learning!

$$Q(s, a) = f_{\theta}(s, a)$$

Neural Networks: Non-linear function approximation



Differences between RL and Supervised Learning

Predicting State-Action Value

Input: (s, a)

Output: $Q_{\theta}(s, a)$

Target: $r + \gamma \max_{a'} Q_{\theta}(s', a')$

Predicting House Price

Input: size, #bedrooms,
nearby school ratings, year
built, etc.

Output: $f_{\theta}(x)$

Target: \$680K

RL has a non-stationary target! This leads to instabilities if using non-linear function approximation.

How to get Q-Learning to work with Deep Learning?

- Experience Replay Buffer

- Don't throw away each transition (s,a,r,s')
- Save them in a buffer or “replay memory”
- During training randomly sample a batch of transitions to update Q

How to get Q-Learning to work with Deep Learning?

- Target Network

- Keep the network for the target fixed and only update periodically

Like before we want to update Q to minimize the error:

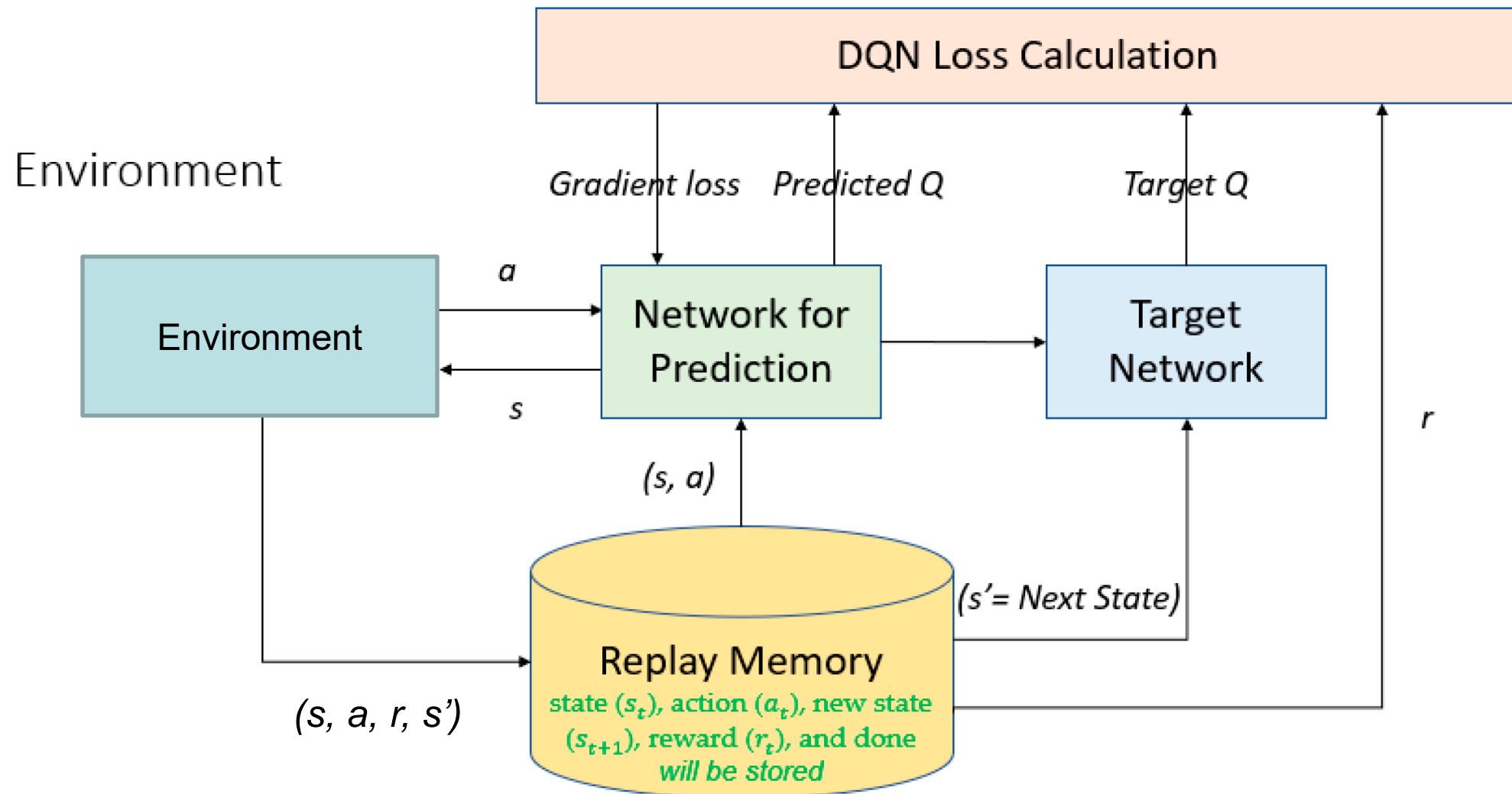
$$error = \frac{1}{2} \left(r + \gamma \max_{a'} Q_T(s', a'; \theta^-) - Q(s, a; \theta) \right)^2$$

$$\nabla_{\theta} error = - \left(r + \gamma \max_{a'} Q_T(s', a'; \theta^-) - Q(s, a; \theta) \right) \nabla_{\theta} Q(s, a; \theta)$$

Take step to decrease error (in the direction of the negative gradient)

$$\theta \leftarrow \theta + \alpha \left(r + \gamma \max_{a'} Q_T(s', a'; \theta^-) - Q(s, a; \theta) \right) \nabla_{\theta} Q(s, a; \theta)$$

Overview of DQN



Deep RL Makes a Big Splash!

nature

Explore content ▾

About the journal ▾

Publish with us ▾

Subscribe

[nature](#) > [letters](#) > article

[Published: 25 February 2015](#)

Human-level control through deep reinforcement learning

[Volodymyr Mnih](#), [Koray Kavukcuoglu](#) , [David Silver](#), [Andrei A. Rusu](#), [Joel Veness](#), [Marc G. Bellemare](#),
[Alex Graves](#), [Martin Riedmiller](#), [Andreas K. Fidjeland](#), [Georg Ostrovski](#), [Stig Petersen](#), [Charles Beattie](#), [Amir](#)
[Sadik](#), [Ioannis Antonoglou](#), [Helen King](#), [Dharshan Kumaran](#), [Daan Wierstra](#), [Shane Legg](#) & [Demis Hassabis](#)





Login

Search Q

TechCrunch+

Startups

Venture

Security

AI

Crypto

Apps

Events

Startup Battlefield

More

Startups

Google Acquires Artificial Intelligence Startup DeepMind For More Than \$500M

Catherine Shu @catherineshu / 6:20 PM MST • January 26, 2014

Comment



TechCrunch
Early Stage

Regi

Ad

WATCH
ALL SEA
LIVE AND

New users only. Valid form
subscription price after trial.

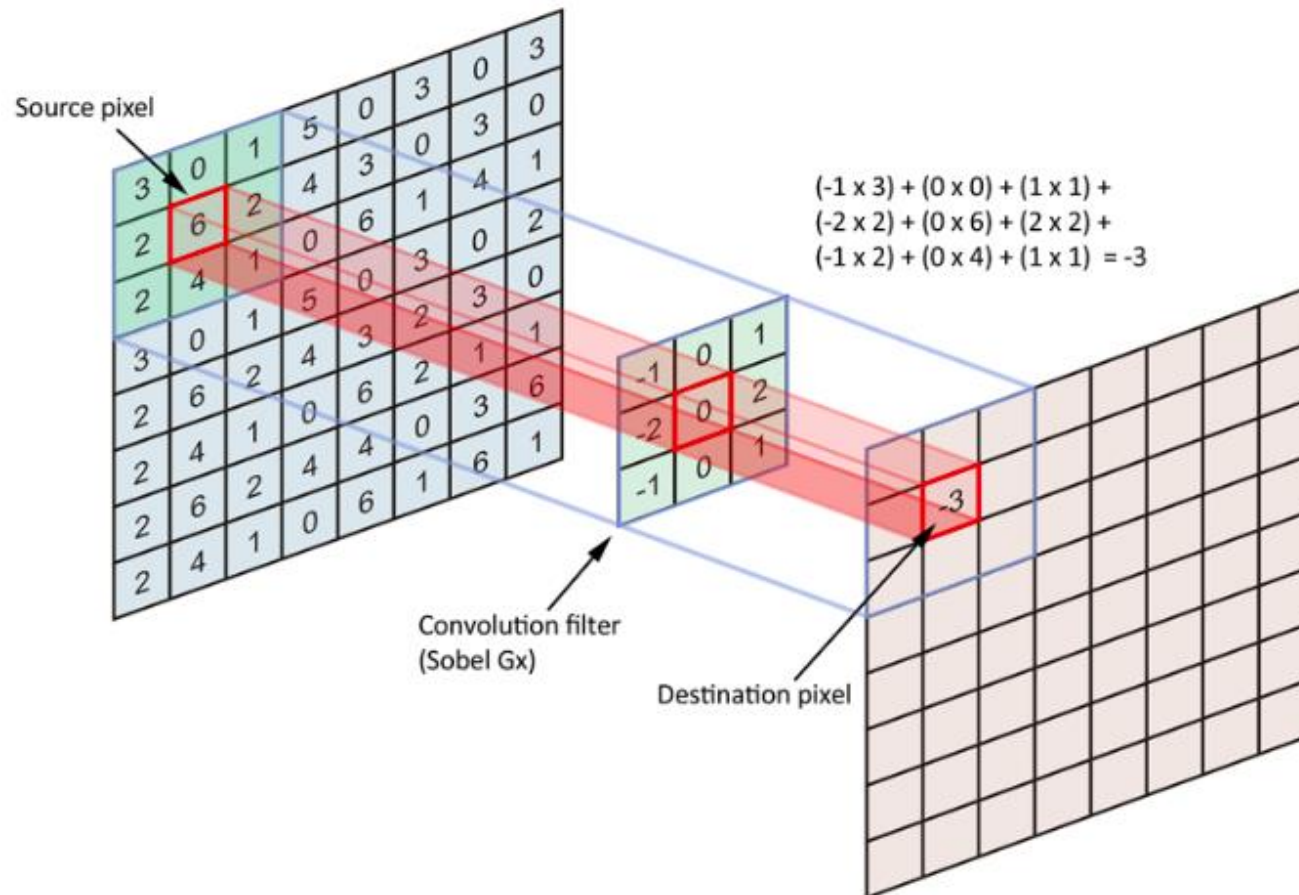
YouTube TV

The Arcade Learning Environment



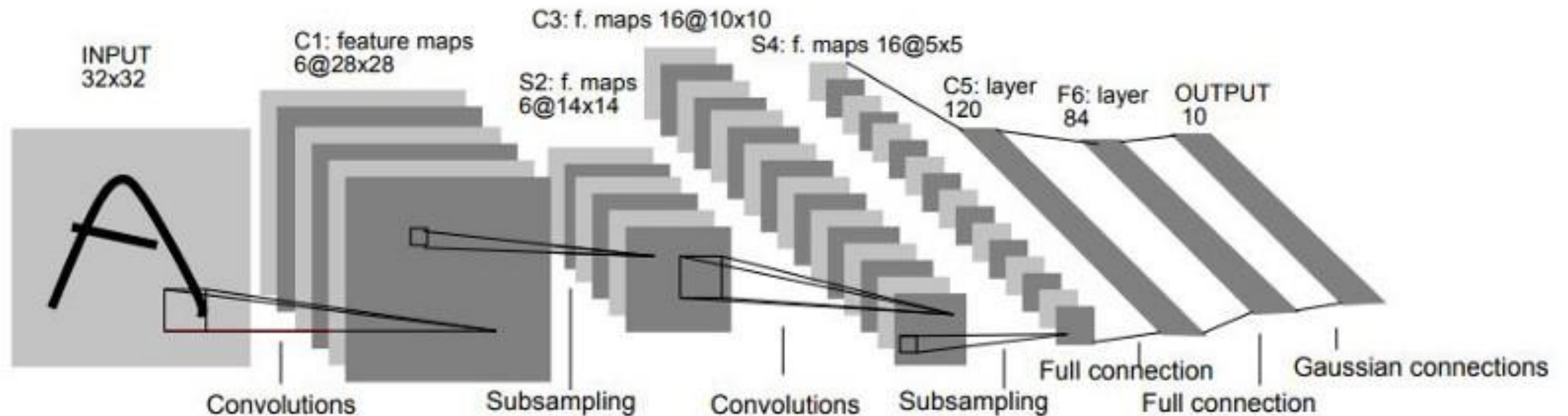
How do you learn from raw pixels?

- Too many parameters to have a weight for each pixel.
- Use a convolutional filter



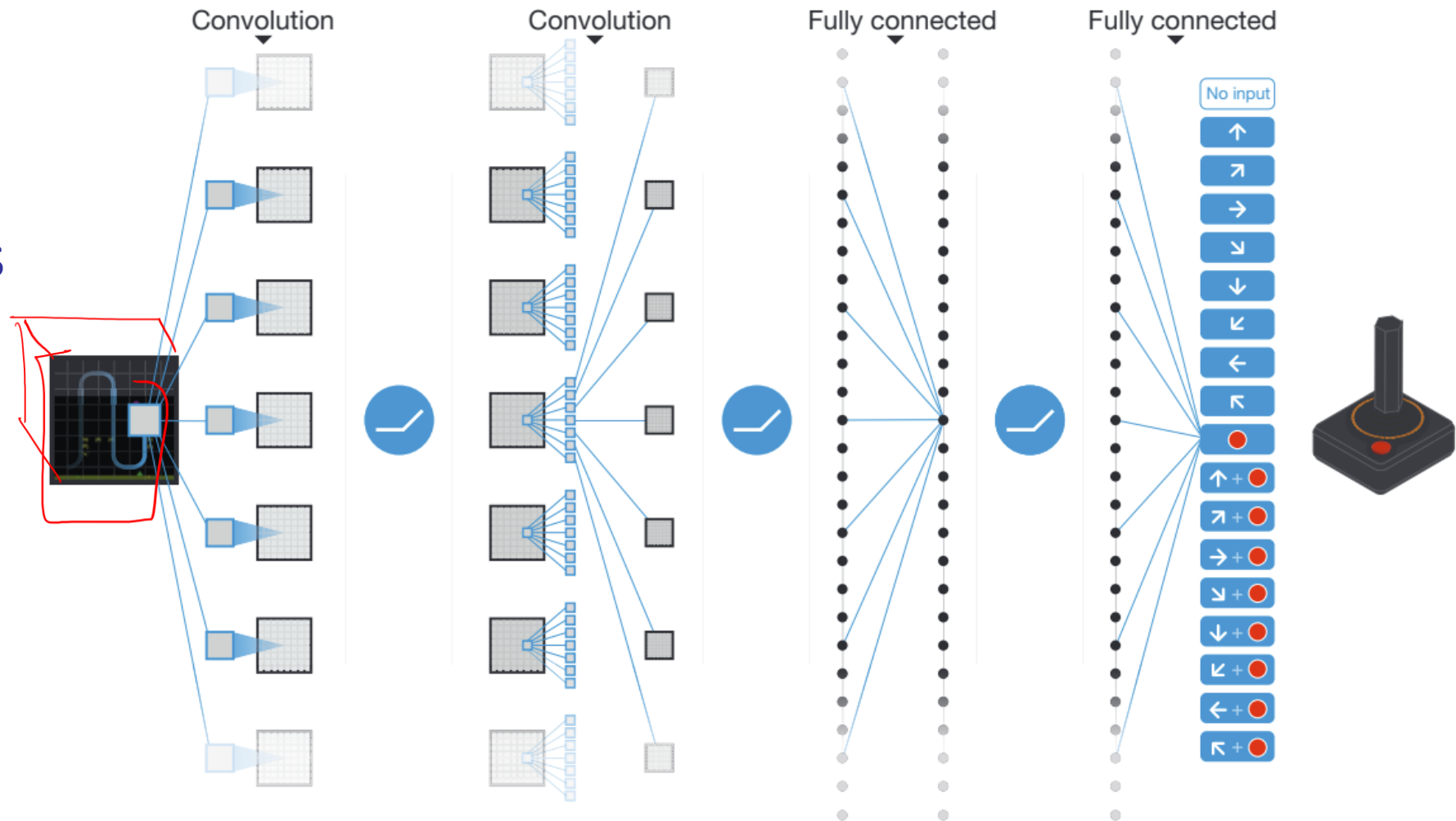
How do you learn from raw pixels?

- Too many parameters to have a weight for each pixel.
- Use a convolutional filter
- Use several layers of multiple filters

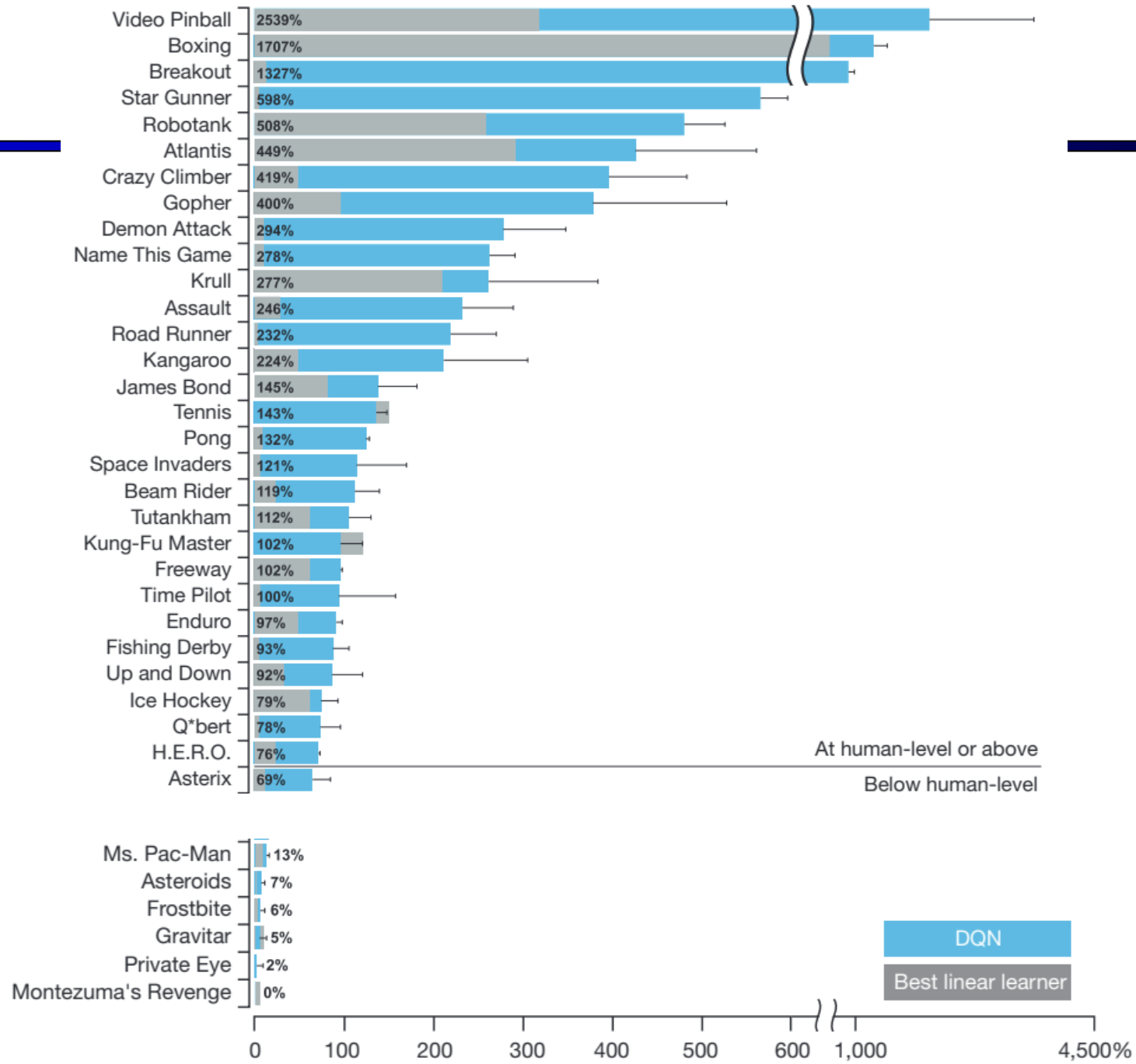


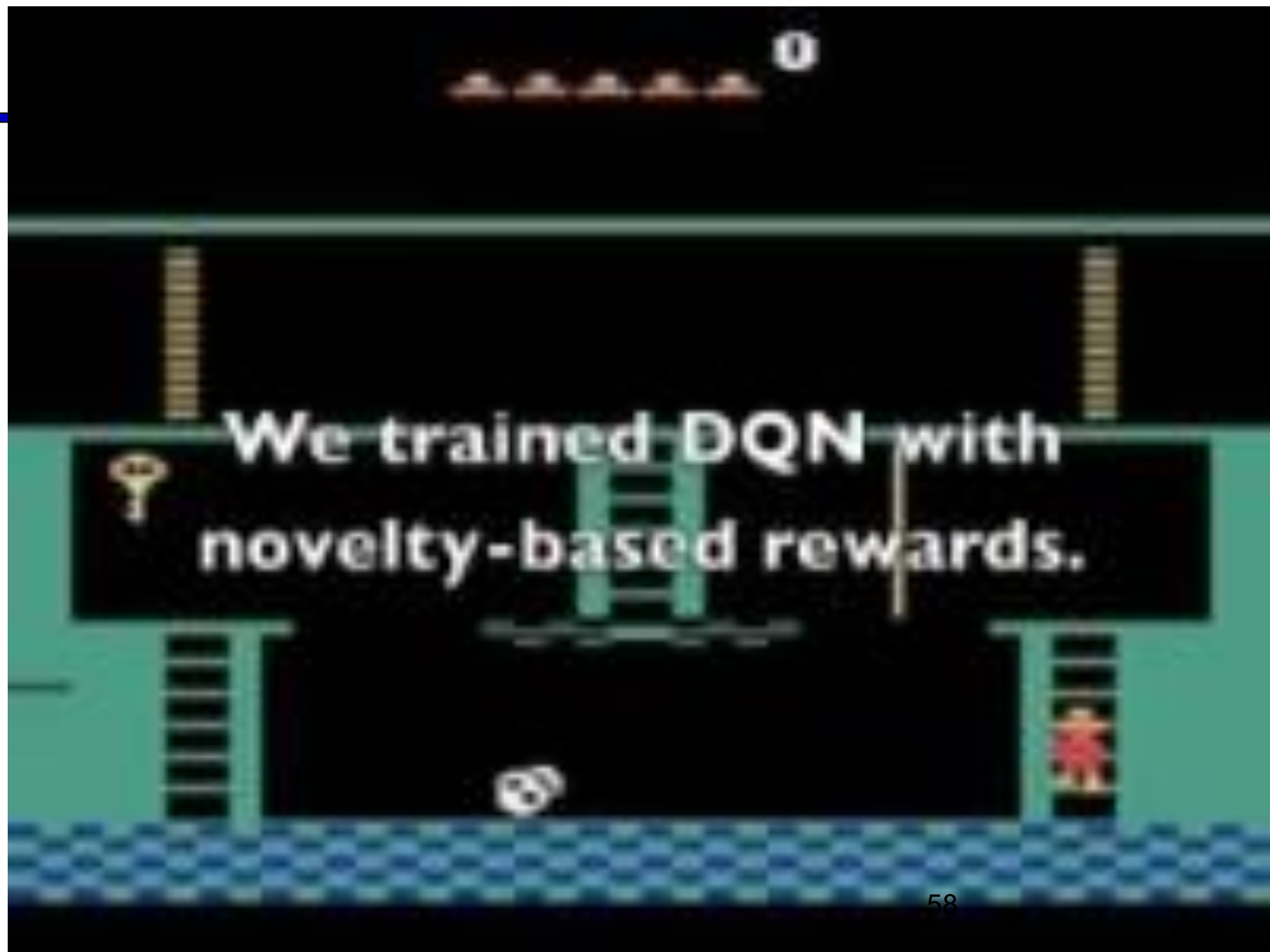
High-Level Architecture

- Learns to “see” through trial and error!
- Learns what actions to take to maximize game score.
- Epsilon-greedy exploration.









DQN only works for discrete action spaces

- Next Time: How to deal with continuous action spaces

